

Inheritance

Inheritance

- Previously, if we needed information of one class in another, we created a composition that involved an all encompassing class
- Inheritance does not replace this, instead it is another tool

Concept

- Inheritance is understood as an "is-a" relationship
 - A Square is a Shape
 - A Cat is a Mammal
 - A Car is a Vehicle

Basic Syntax and Structure

- Inheritance is declared by passing the name of the **parent class** as a parameter into the **child class or subclass's** definition

```
class Vehcile:

    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

class Car(Vehicle):
    def __init__(self, brand, model, door_count):
        super().__init__(brand, model)
        self.door_count = door_count
```

Basic Syntax and Structure (contd.)

- `super()` - built-in method used to access and call methods from a parent class
 - often used in `__init__`
 - standard practice to call the **parent** `__init__` in child class `__init__`
 - pass in appropriate parameters into the method as well

Method Overriding and Specialization

- **Overriding** a method occurs when a **subclass** needs to alter the **parent class** method
- utilizes the same name
- can call upon parent class method if necessary and alter those results
- Does not necessarily need to replace **parent** method